

Value vs Reference

Primitives copy the value. Objects copy the reference: two names, one object.

Primitives copy the VALUE

```
let a = 10;  
let b = a; // copies the number  
b = 20;  
// a is still 10
```

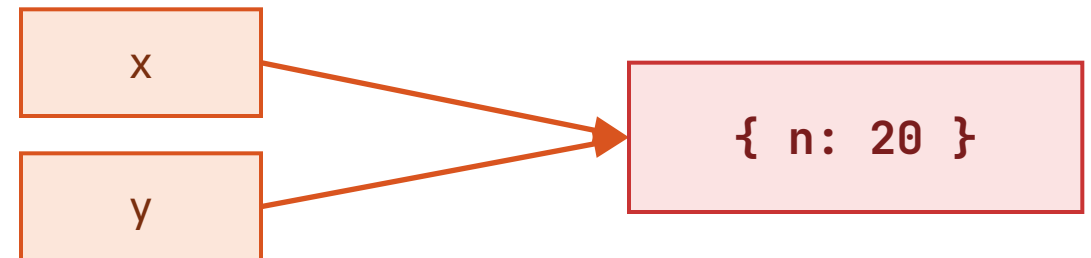
a → 10

b → 20

Two separate boxes. Changing one cannot touch the other. Same for string, boolean, number, null, undefined, symbol, bigint.

Objects copy the REFERENCE

```
const x = { n: 10 };  
const y = x; // copies the arrow, not the box  
y.n = 20;  
// x.n is now 20 too
```



One object, two names. Arrays and functions are objects too, so they behave the same way.

To get a real copy, spread it: `const z = { ...x }` or `const arr2 = [ ...arr ]`
But spread is SHALLOW: a nested object is still shared. Use `structuredClone(x)` for a deep copy.
Also why `const` does not mean immutable: it fixes the arrow, not the box it points at.